

Instance Segmentation

1. Experiment Objective

Use YOLO for instance segmentation — segment object contours at the pixel level and distinguish different instances of the same class. It supports three input sources: images, videos, and a real-time camera feed.

2. Experiment Principle

1. Glossary

Term	Description
Instance Segmentation	The combination of object detection and semantic segmentation, outputting both bounding boxes and pixel-level masks
Mask	A mask image of the same size as the original image (binary or grayscale) that marks whether each pixel belongs to the target object
Ultralytics	The official Python library for YOLO models; instance segmentation models are identified by the <code>-seg</code> suffix (e.g. <code>yolo11n-seg.pt</code>)
COCO	A general-purpose vision dataset; instance segmentation tasks are trained with COCO's 80 classes plus pixel-level annotations
mAP	mean Average Precision; in instance segmentation, both bounding box accuracy (box mAP) and mask accuracy (mask mAP) are evaluated
Contour	The set of pixel points along the object edge, i.e. the outer boundary of an instance segmentation mask

2. Technical Architecture

YOLO instance segmentation starts inference nodes through ROS2 launch files, supporting three input sources:

- **Image/Video mode:** The node reads local files for offline inference; it does not require `micro_ros_agent` or the state estimation layer.
- **Real-time camera mode:** The node subscribes to the `/camera/image_raw` topic to get real-time camera frames. It requires three processes to work together: `micro_ros_agent` → state estimation layer → YOLO segmentation node.

Real-time mode data flow:

```
ESP32 camera → micro_ros_agent → /camera/image_raw → YOLO segmentation node → /yolo_seg/annotated_image + /yolo_seg/status
```

YOLO inference depends on the ultralytics + PyTorch libraries installed in the current Python environment; the ROS2 and workspace environment must be sourced before starting.

3. Core Topic Quick Reference

ROS2 Internal Topics

Topic	Message Type	QoS	Source	Description
<code>/camera/image_raw</code>	<code>sensor_msgs/Image</code>	best_effort	State estimation layer	Raw camera image, subscribed by the YOLO segmentation node
<code>/yolo_seg/annotated_image</code>	<code>sensor_msgs/Image</code>	reliable	YOLO segmentation node	Annotated segmentation result image (with mask overlay)
<code>/yolo_seg/status</code>	<code>std_msgs/String</code>	reliable	YOLO segmentation node	Segmentation status info (FPS, detection count, mask count)

3. Experiment Steps

1. Hardware Requirements

Hardware	Requirement
PTZ version	☒ Supported
No-PTZ version	☒ Supported
Required peripheral	ESP32 camera (realtime mode)

2. Start the YOLO Environment

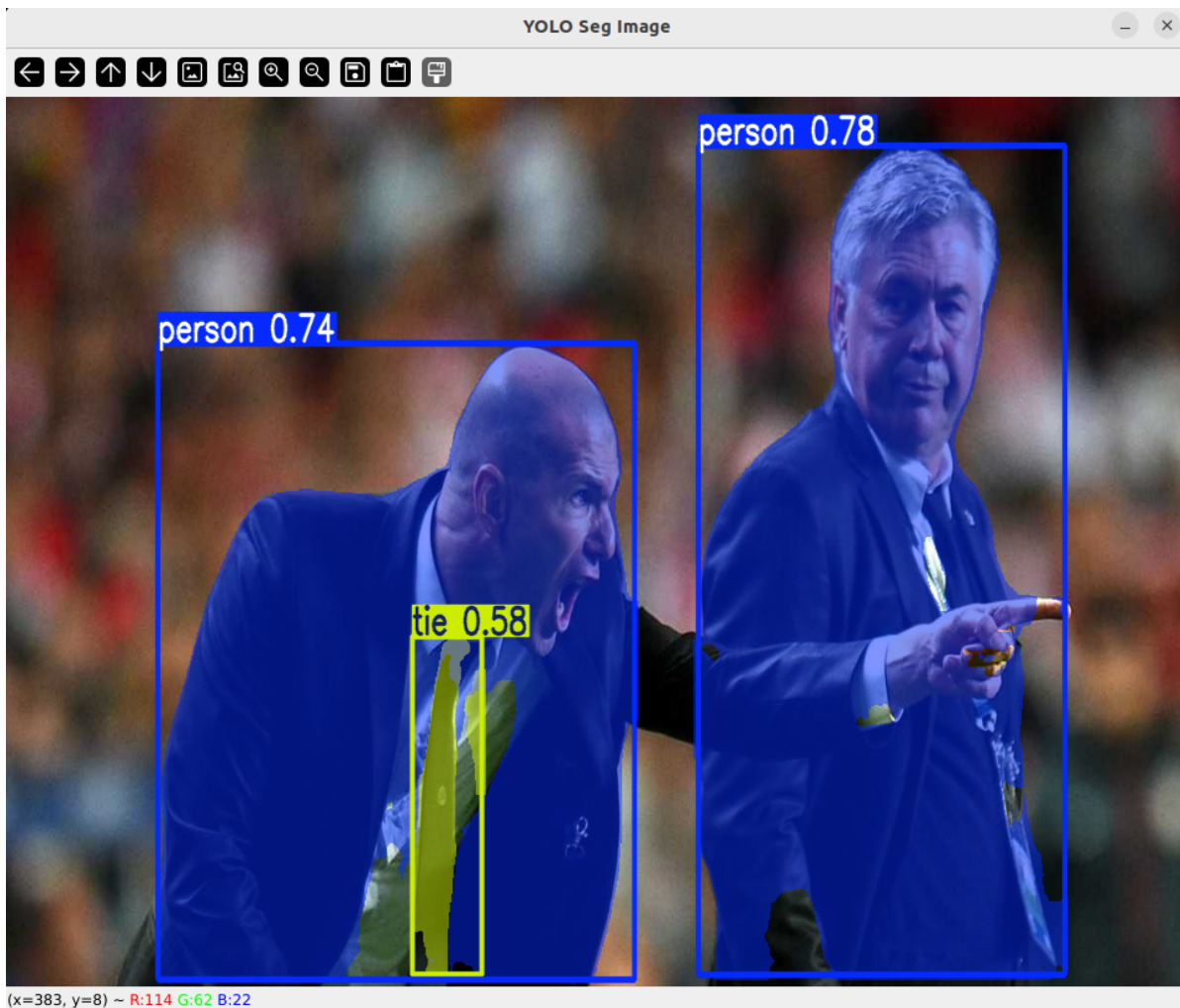
All YOLO ROS2 nodes depend on the ultralytics + PyTorch packages; the environment must be activated before starting:

```
yolo_env
```

3. Image Mode (Single Image Inference)

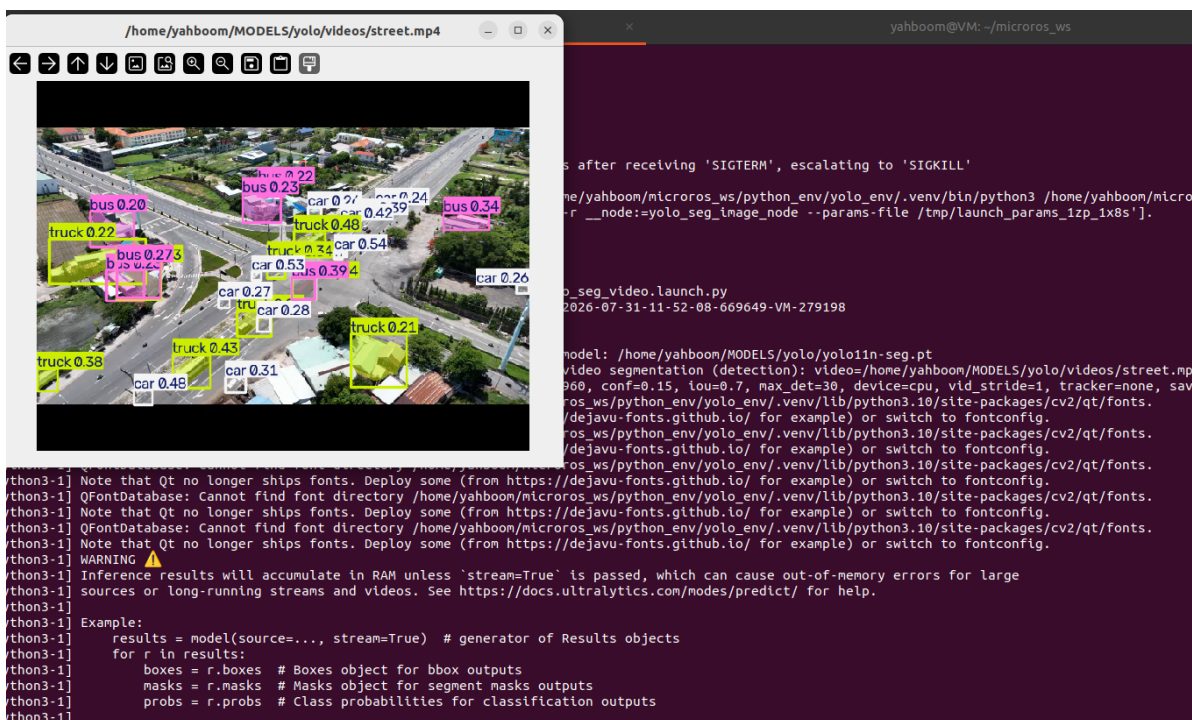
```
ros2 launch microros_v2_yolo_pkg yolo_seg_image.launch.py
```

```
(yolo_env) yahboom@VM: /microros_ws$ ros2 launch microros_v2_yolo_pkg yolo_seg_image.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-31-11-48-34-492260-VM-274880
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [python3-1]: process started with pid [274901]
[python3-1] [INFO] [1785469716.443770259] [yolo_seg_image_node]: Loading model: /home/yahboom/MODELS/yolo/yolo11n-seg.pt
[python3-1] [INFO] [1785469716.712794608] [yolo_seg_image_node]: Running image segmentation: image=/home/yahboom/MODELS/yolo/assets/zidane.jpg, imgsz=960, conf=0.05, 7, max_det=5, device=cpu
[python3-1] [INFO] [1785469730.920073861] [yolo_seg_image_node]: Detected boxes=5, masks=5
[python3-1] [INFO] [1785469731.117624993] [yolo_seg_image_node]: Saved output image: /home/yahboom/MODELS/yolo/assets/output/zidane_yolo11n-seg_seg_imgs960_conf0p05.jpg
[python3-1] QFontDatabase: Cannot find font directory /home/yahboom/microros_ws/python_env/yolo_env/.venv/lib/python3.10/site-packages/cv2/qt/fonts.
[python3-1] Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for example) or switch to fontconfig.
[python3-1] QFontDatabase: Cannot find font directory /home/yahboom/microros_ws/python_env/yolo_env/.venv/lib/python3.10/site-packages/cv2/qt/fonts.
[python3-1] Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for example) or switch to fontconfig.
[python3-1] QFontDatabase: Cannot find font directory /home/yahboom/microros_ws/python_env/yolo_env/.venv/lib/python3.10/site-packages/cv2/qt/fonts.
[python3-1] Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for example) or switch to fontconfig.
[python3-1] QFontDatabase: Cannot find font directory /home/yahboom/microros_ws/python_env/yolo_env/.venv/lib/python3.10/site-packages/cv2/qt/fonts.
[python3-1] Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for example) or switch to fontconfig.
[python3-1] QFontDatabase: Cannot find font directory /home/yahboom/microros_ws/python_env/yolo_env/.venv/lib/python3.10/site-packages/cv2/qt/fonts.
[python3-1] Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for example) or switch to fontconfig.
[python3-1] [INFO] [1785469732.100142188] [yolo_seg_image_node]: Press any key in the OpenCV window to close it.
```



4. Video Mode (Frame-by-Frame Inference on a Video File)

```
ros2 launch microros_v2_yolo_pkg yolo_seg_video.launch.py
```



5. Real-time Camera Mode

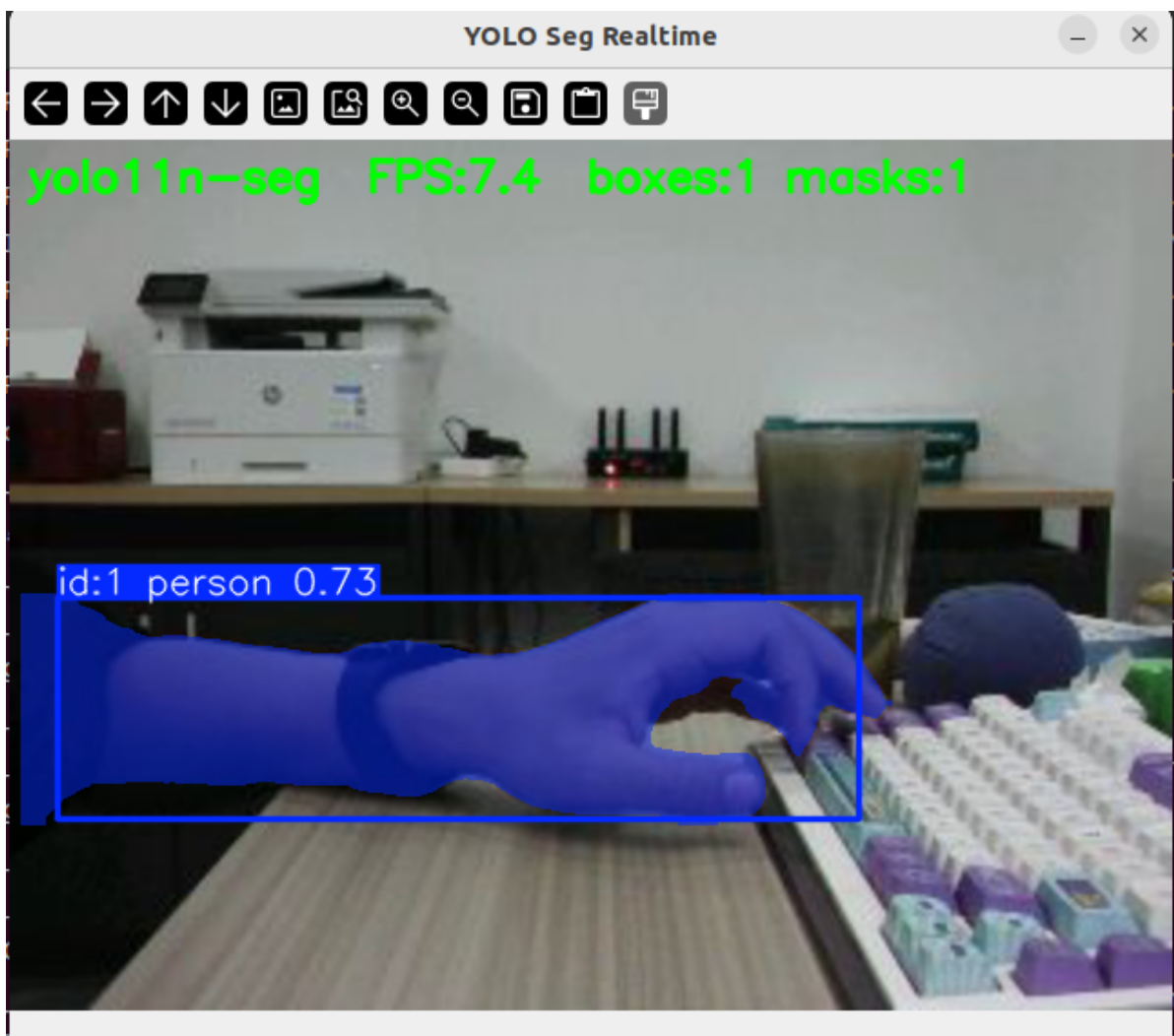
Launch the proxy + camera + joint model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic..
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | Init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment can1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment can2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment can3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

Terminal 2 — Launch YOLO real-time segmentation:

```
ros2 launch microros_v2_yolo_pkg yolo_seg_realtime.launch.py
```



6. Operating Instructions

- **Image mode:** The program reads a single image and overlays a **colored semi-transparent mask** on the object contours after inference. Each mask is accompanied by a bounding box, class name, and confidence. Press `q` or `ESC` to close the window and exit.
- **Video mode:** Frame-by-frame inference with mask annotation; the masked regions update in real time as objects move. Different instances of different classes are distinguished by different colors. Press `q` or `ESC` to stop playback and exit.
- **Real-time camera mode:** Subscribes to the camera topic for real-time inference; the model name and live FPS are shown at the top of the frame. ByteTrack object tracking is enabled by default (`use_track=true`) to keep the same target ID across frames. Press `q` or `ESC` to exit.

Instance segmentation is finer-grained than object detection — object detection only outputs rectangular boxes, while instance segmentation outputs the precise contour of each object (the edge hugs the object shape). In scenes where objects occlude each other, segmentation clearly outperforms detection boxes.

7. How to Stop

Press `Ctrl+C` in each terminal, or press `q` or `ESC` to exit.

4. Experimental Phenomena

1. Image Mode

- Each object in the frame is precisely covered by a colored semi-transparent mask whose contour hugs the object shape.
- The top-left corner of the frame shows the model name, real-time FPS, detection box count, and mask count.
- Different classes are distinguished by different mask colors.

2. Video Mode

- Every video frame outputs precise object-contour masks, and the covered regions update in near-real time.
- For fast-moving objects, masks may show slight lag or jitter (a CPU-inference limitation).

3. Real-time Camera Mode

- Objects in the camera feed are covered by colored masks; tracking is enabled by default, supporting stable cross-frame IDs.
- When objects occlude each other (e.g. several stacked bottles), segmentation clearly outperforms plain detection boxes.

4. Edge Cases

- Segmentation accuracy for small objects is limited (smaller than 32x32 pixels); valid masks may not be produced.
- Transparent/reflective objects may be segmented poorly, limited by the training data distribution.

5. Program Analysis

Source code paths:

- Launch (real-time):
`~/microros_ws/src/microros_v2_yolo_pkg/launch/yolo_seg_realtime.launch.py`
- Launch (image):
`~/microros_ws/src/microros_v2_yolo_pkg/launch/yolo_seg_image.launch.py`
- Launch (video):
`~/microros_ws/src/microros_v2_yolo_pkg/launch/yolo_seg_video.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_yolo_pkg/microros_v2_yolo_pkg/yolo_seg_realtime_node.py`

1. Core Node Analysis

The real-time segmentation node `yolo_seg_realtime_node` has an architecture similar to the real-time detection node. The key differences are that it uses a segmentation model (`-seg.pt`) and publishes segmentation-specific topics (`/yolo_seg/annotated_image`, `/yolo_seg/status`). The node subscribes to the raw image topic, calls the YOLO segmentation model for inference, and publishes the annotated results and status.

ROS2 communication setup: It subscribes to the raw image and publishes the annotated image and status; the QoS can be configured through launch parameters (default `best_effort`). The node uses `image_callback` to receive `sensor_msgs/Image` and drops new frames while CPU inference is busy to keep latency low.

```
# source file:
~/microros_ws/src/microros_v2_yolo_pkg/microros_v2_yolo_pkg/yolo_seg_realtime_node.py

def __init__(self) -> None:
    super().__init__("yolo_seg_realtime_node")

    self.declare_parameter("model_path", DEFAULT_MODEL)
    self.declare_parameter("image_topic", "/camera/image_raw")
    self.declare_parameter("annotated_topic", "/yolo_seg/annotated_image")
    self.declare_parameter("status_topic", "/yolo_seg/status")
    self.declare_parameter("imgsz", 960)
    self.declare_parameter("conf", 0.05)
    self.declare_parameter("iou", 0.7)
    self.declare_parameter("max_det", 1000)
    self.declare_parameter("device", "cpu")
    self.declare_parameter("tracker", "bytetrack.yaml")
    self.declare_parameter("show", True)
    self.declare_parameter("save_video", True)
    self.declare_parameter("output_fps", 20.0)
    self.declare_parameter("window_width", 1280)
    self.declare_parameter("window_height", 720)
    self.declare_parameter("qos_reliability", "best_effort")
    self.declare_parameter("qos_depth", 1)
    self.declare_parameter("process_every_n_frames", 1)
    self.declare_parameter("line_width", 2)
    self.declare_parameter("retina_masks", False)
    self.declare_parameter("publish_status", True)

    self.model_path = str(self.get_parameter("model_path").value).strip()
    self.image_topic = str(self.get_parameter("image_topic").value).strip()
```

```

        self.annotated_topic =
str(self.get_parameter("annotated_topic").value).strip()
        self.status_topic =
str(self.get_parameter("status_topic").value).strip()
        self.output_dir = str(self.get_parameter("output_dir").value).strip()
        self.imgsz = int(self.get_parameter("imgsz").value)
        self.conf = float(self.get_parameter("conf").value)
        self.iou = float(self.get_parameter("iou").value)
        self.max_det = int(self.get_parameter("max_det").value)
        self.device = str(self.get_parameter("device").value).strip()
        self.tracker = str(self.get_parameter("tracker").value).strip()
        self.show = _as_bool(self.get_parameter("show").value)
        self.save_video = _as_bool(self.get_parameter("save_video").value)
        self.output_fps = float(self.get_parameter("output_fps").value)
        self.window_width = int(self.get_parameter("window_width").value)
        self.window_height = int(self.get_parameter("window_height").value)
        self.qos_reliability =
str(self.get_parameter("qos_reliability").value).strip()
        self.qos_depth = int(self.get_parameter("qos_depth").value)
        self.process_every_n_frames = max(1,
int(self.get_parameter("process_every_n_frames").value))
        self.line_width = int(self.get_parameter("line_width").value)
        self.retina_masks = _as_bool(self.get_parameter("retina_masks").value)
        self.publish_status =
_as_bool(self.get_parameter("publish_status").value)

        self.annotated_pub = self.create_publisher(Image, self.annotated_topic,
10)
        self.status_pub = self.create_publisher(String, self.status_topic, 10)
        if self.publish_status else None

        qos = _make_qos(self.qos_reliability, self.qos_depth)
        self.sub = self.create_subscription(Image, self.image_topic,
self.image_callback, qos)

```

Image callback (`image_callback`): Receives raw image frames, calls `model.track()` for inference, obtains masks, draws the annotations on the frame, and publishes the `Image` result and the optional `String` status. The `processing` flag prevents callbacks from stacking up during CPU inference.

```

# source file:
~/microros_ws/src/microros_v2_yolo_pkg/microros_v2_yolo_pkg/yolo_seg_realtime_no
de.py

def image_callback(self, msg: Image) -> None:
    self.frame_index += 1
    if self.frame_index % self.process_every_n_frames != 0:
        return
    if self.processing:
        return

    self.processing = True
    try:
        frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
        results = self.model.track(
            source=frame, persist=True, tracker=self.tracker,

```

```

        imgsz=self.imgsz, conf=self.conf, iou=self.iou,
        max_det=self.max_det, device=self.device,
        line_width=self.line_width, retina_masks=self.retina_masks,
        verbose=False,
    )
    if not results:
        return
    annotated = results[0].plot()
    out_msg = self.bridge.cv2_to_imgmsg(annotated, encoding="bgr8")
    out_msg.header = msg.header
    self.annotated_pub.publish(out_msg)
finally:
    self.processing = False

```

2. Key Parameters

Parameter definition files:

- Launch:


```
~/microros_ws/src/microros_v2_yolo_pkg/launch/yolo_seg_realtime.launch.py
```
- Node:


```
~/microros_ws/src/microros_v2_yolo_pkg/microros_v2_yolo_pkg/yolo_seg_realtime_node.py
```


Parameter	Type	Default	Description
<code>model_path</code>	string	<code>.../models/yolo/yolo11n-seg.pt</code>	Absolute path of the segmentation model
<code>imgsz</code>	int	960	Inference input image size (pixels)
<code>conf</code>	float	0.05	Detection confidence threshold; results below this value are filtered
<code>iou</code>	float	0.7	IoU threshold for NMS
<code>max_det</code>	int	1000	Maximum number of detected objects per frame
<code>device</code>	string	<code>cpu</code>	Inference device; <code>cpu</code> or <code>cuda:0</code>
<code>tracker</code>	string	<code>bytetrack.yaml</code>	Tracker configuration file
<code>line_width</code>	int	2	Pixel width of bounding boxes and contour lines
<code>retina_masks</code>	bool	<code>false</code>	Whether to output masks at the original image resolution (higher accuracy but slower)
<code>show</code>	bool	<code>true</code>	Whether to show the OpenCV inference window
<code>window_width</code>	int	1280	Display window width
<code>window_height</code>	int	720	Display window height
<code>save_video</code>	bool	<code>true</code>	Whether to save the output video
<code>output_fps</code>	float	20.0	Frame rate of the saved video
<code>process_every_n_frames</code>	int	1	Process every N frames (frame-skipping inference)

Parameter	Type	Default	Description
<code>publish_status</code>	bool	<code>true</code>	Whether to publish the status topic <code>/yolo_seg/status</code>

3. Node Description

Node	Executable Name	Description
<code>yolo_seg_realtime_node</code>	<code>yolo_seg_realtime_node</code>	Real-time segmentation node; subscribes to <code>/camera/image_raw</code> and publishes <code>/yolo_seg/annotated_image</code> + <code>/yolo_seg/status</code>
<code>yolo_seg_image_node</code>	<code>yolo_seg_image_node</code>	Image segmentation node; reads a local image and outputs mask-annotated results
<code>yolo_seg_video_node</code>	<code>yolo_seg_video_node</code>	Video segmentation node; reads a video frame by frame and outputs a video annotated with masks

6. Common Problems

Problem	Cause	Solution
Segmentation mask does not hug the object edge	Model accuracy limit or input resolution too low	Raise <code>imgsz</code> (e.g. <code>imgsz:=1280</code>) or enable <code>retina_masks:=true</code>
Holes in the segmentation result (hollow masks)	The model branch does not fully cover the inside of the object	This is a model limitation; switch to a larger model such as <code>yolo11m-seg</code>
Small objects cannot be segmented	Too few pixels for the model to generate a valid mask	Get closer to the object or use a higher-resolution camera
Real-time FPS too low	The segmentation model needs more computation than the detection model	Switch to a lightweight model, lower <code>imgsz</code> , or increase <code>process_every_n_frames</code>
<code>ModuleNotFoundError: ultralytics</code>	The current Python environment lacks the dependencies	Run <code>python3 -m pip install ultralytics torch torchvision opencv-python</code> , or activate an existing virtual environment first

